TECHNICAL REPORT

Stage 3

University of Cape Town
CSC3002F | Networks & Operating Systems

Contents

1. Architecture Overview 2
2. Protocol Specification and System Functions 3
Message Framing 3
Frame Structure 3
Message types 4
Authentication Messages 4
Data Messages 4
Control Messages 5
3. WIRESHARK: Network Protocol Analyzer 6
2.2 Implementation of Key System Functions 6
Authentication 6
One-to-One Messaging 7
Group Chat 7
Media Transfer (Hybrid Architecture) 7
4. Implementation Changes 8
1. Designing a Custom Message Framing Protocol 8
2. Changes to the Designed Architecture 8
5. Performance and Reliability Discussion 9
6. Basic Security Consideration 10
7. Use of Generative AI 10

1. Architecture Overview

The document describes the design, implementation and evaluation of a networked chat application called **C00NECTED**.

C00NECTED is a networked chat application built around a hybrid client-server and peer-to-peer(P2P) architecture. The system separates concerns across three distinct layers:

- Client layer responsible for user interaction,
- Server layer responsible for authentication, message routing
- Storage layer responsible for persisting user accounts, group data and media.

The peer-to-peer communication is layered on top of this for file transfer and real-time calls.

Component	Role
ARCPServer	• Entry point for all client connection. • Handles authentication, message framing, command routing, media storage, offline queue flushing
ChatServer	• Core server logic. Manages the in-memory client registry, group membership, real-time messaging, and peer signalling. • Delegates group creation, last-seen timestamps, and RAM management to SQLite, and user presence tracking to FakeRedis.
Client	• Handles Authentication, peer discovery, direct messaging, media transfers, group chat management, message history and queuing via TCP to ARCPServer. • Supports direct UDP calls to peer clients
FakeRedis	• Store online user presence keys (IP, TCP, UDP call port) managed by ChatServer, and offline message queues (queue and flush) by ARCPServer
SQLite	• Persistent storage for user registration and authentication data, chat history, media blobs, group creation and membership records and last-seen timestamps.

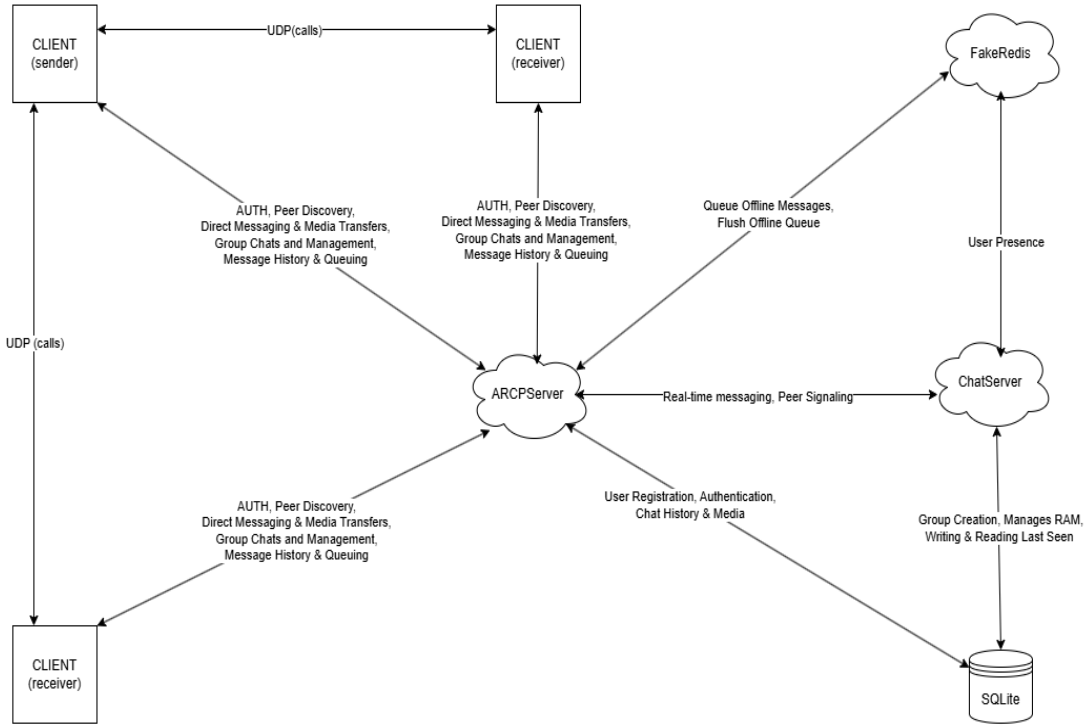


Figure 1: Final System Architecture Diagram

2. Protocol Specification and System Functions

Our chat system **C00NECTED** has a custom application layer protocol called **ALL- Round Connect Protocol (ARCP)**. The protocol defines how the client and server communicate, the structure of messages, and how major system functions such as login, messaging, group chats, and media transfer work.

Message Framing

A message sent by the client may arrive in multiple parts as TCP is a byte-stream protocol-does not preserve message boundaries. To solve this problem, ARCP uses a 9-byte header before every message, so the receiver knows exactly how many bytes to read.

Frame Structure

Each message contains

- **Type (1 byte)**- A single ASCII character classifying the message. Three types defined:
  - A(Authentication): Login, registration and credential exchange
  - D(data): Chat message delivery and file/ media content
  - C(Control): System command, server responses, peer discovery, group management, call signalling and status notifications.
- **Length (8 bytes)**- size of payload
- **Payload**-the actual message content

Message types

ARCP defines several message categories.

Authentication Messages

Messages used during login and registration.

Message	Description
CHECK:<username>	Query whether a username is registered before attempting login or registration
LOGIN:<username>:<password>	Authenticate with existing credentials. Password is compared against the stored salted SHA-256 hash.
REG:<username>:<password>	Register a new account. The server validates password strength before hashing and storing.
SUCCESS	— Authentication or registration succeeded. Client must now send PORT: and CALL_PORT:.
FAIL	Incorrect password
USER_EXISTS	Registration rejected: username already taken.
WEAK_PASSWORD	Password rejected. Reasons: too short, missing uppercase, lowercase, digit, special character, or matches a known weak password.

Data Messages

These messages carry chat messages and media content

Message	Description
SEND:<recipient>:<message_text>	Send a direct message to another registered user.
SEND_GROUP:<group_id>:<message_text>	Send a message to all members of a group.
UPLOAD_MEDIA	Upload a file for offline storage or group forwarding. Target may be a username or group ID.
FILE:<filename>:<filetype>:<base64_data>:<sender>	Server delivers a stored media file directly to the client's socket, either for online forwarding or on reconnect after offline queuing.
GET_HISTORY:<chat_id>:<since_timestamp>	Request server-side message history for a conversation since a given timestamp. New in final version — enables cross-device message sync.
HISTORY:<chat_id>:<json_array>	Server responds with a JSON array of up to 500 stored messages

Timestamped message	[YYYY-MM-DD HH:MM:SS] [sender (DM)]: content, Delivered as a Type D frame to the recipient's socket when they are online.

Control Messages

Control messages manage system actions.

Message	Description
CREATE_GROUP:<group_id>:	Create a new group. The sender is automatically added as the first member.
ADD_TO_GROUP:<group_id>:<target_username>	Add a user to an existing group. Only existing members may add others
LEAVE_GROUP:<group_id>:	Remove the sender from the group. If the last member leaves, the group is deleted from SQLite.
GET_PEER:<target>:<filename>	Request the IP and TCP media port of a peer or group for P2P file transfer.
AUDIO_CALL:<recipient>	Request the recipient's UDP call port to initiate an audio cal
EXIT	Server closes the socket and removes the client from the registry, disconnect.
ERROR: <description>	Returned for any invalid command, unknown user, missing group, bad format, or failed operation

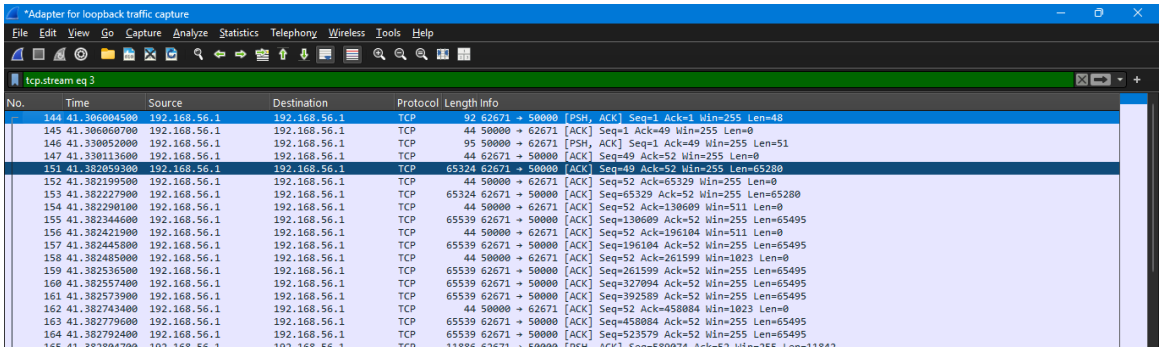
3. WIRESHARK: Network Protocol Analyzer

To guarantee the reliable transmission of data over continuous TCP streams, we designed and implemented a custom application-layer protocol called **ARCP**. Because TCP does not preserve message boundaries, ARCP utilizes a strict 9-byte framing header for every packet to prevent fragmentation and concatenation errors.

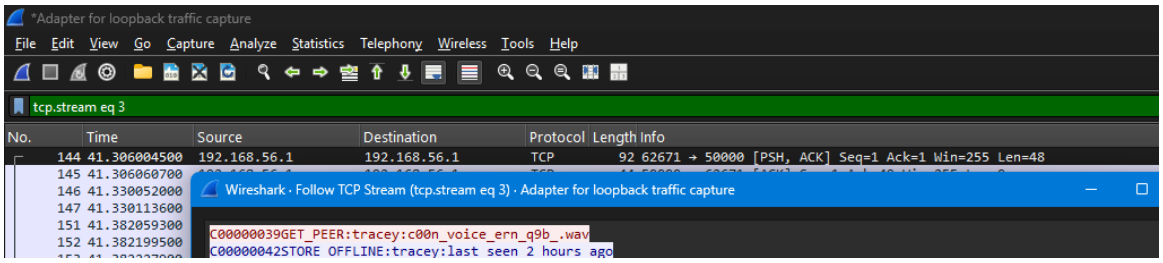
The Header Structure: [1 ASCII CHARACTER] +[length (8 decimal digits)]

- Type Byte: Indicates the classification of the message
  A = Authentication (e.g CHECK,LOGIN,REG)
  C = Control/Signalling (e.g PEER INFO, CALL_RINGING, GROUP_PEER_INFO)

- Length Byte: An 8-digit zero-padded string indicating the exact byte length. The decision to use an 8-digit header (rather than a standard 4-digit header) was a deliberate to allow the system to buffer and transmit massive Base64-encoded media files (up to 99 Megabytes) without overflowing the header capacity.



As an example, our port number was hard-coded for us to be able to pinpoint it on wireshark.
On our C00NECTED chat application the client sent a voice note to the use “tracey”, the packet was picked up by wireshark as seen on the picture. The first light blue line is the packet that was capturing the voice note.



On the second picture after open the TCP Stream option for that packet. It can be seen that header has a type byte and the length byte, together with the recipient’s name and the voice note that was sent.

2.2 Implementation of Key System Functions

Authentication

When a client application launches, it establishes a TCP connection to the central server. The login sequence utilizes a challenge-response flow:

1. The client sends CHECK:<username> to verify existence.
2. The client transmits LOGIN:<username>:<password>. Passwords are not stored in plaintext; the server verifies the credentials against an SQLite database using SHA-256 hashing with a unique 16-byte random salt and timing-safe HMAC comparisons.
3. Upon success, the client dynamically binds a TCP listener (for incoming P2P media) and a UDP socket (for calls). It registers these ports with the server using PORT:<tcp_port> and CALL_PORT:<udp_port>.
4. Finally, the server flushes any queued offline text messages from Redis and pushes them to the client.

One-to-One Messaging

Direct messaging relies on the central server as a message router. The sender transmits SEND:<recipient>:<message>. The server queries the Redis presence dictionary. If the recipient is online, the server immediately forwards the framed message over the recipient's TCP socket. If the recipient is offline, the server pushes the message into a Redis list (offline:<recipient>), which acts as an asynchronous waiting queue until the user's next login.

Group Chat

Group messaging utilizes a fan-out routing mechanism. When a client sends SEND_GROUP:<group_id>:<message>, the server queries the SQLite group_members table to retrieve all associated users. The server iterates through this list, skipping the sender. For every online member, the server forwards the message via their active TCP socket. For offline members, the message is seamlessly diverted into their individual Redis offline queues.

Media Transfer (Hybrid Architecture)

Media transfer (images, PDFs, audio, video) is the most complex subsystem, employing a Hybrid "Store-and-Forward" P2P architecture to ensure delivery regardless of the recipient's network status:

1. **Peer Discovery:** The sender asks the server for the recipient's IP and TCP media port (GET_PEER:<recipient>:<filename>).
2. **Online Scenario (Pure P2P):** If the recipient is online, the server responds with PEER_INFO:<ip>:<port>. The sender's client bypasses the server entirely, opens a direct TCP socket to the recipient, and streams the raw binary file.
3. **Offline Scenario (Store-and-Forward):** If the recipient is offline, the server replies with STORE_OFFLINE. The sender's client automatically converts the binary file into a Base64-encoded text string and uploads it to the server (UPLOAD_MEDIA). The server stores this payload in the SQLite media table and queues a notification. When the recipient logs in, they receive a MEDIA_WAITING alert, prompting their client to automatically download and decode the file from the server.

4. Implementation Changes

1. Designing a Custom Message Framing Protocol

- TCP is a stream protocol; it has no built-in concept of message boundaries.
- When one side calls *sock.sendall(data),* the other side might receive it in one chunk, two chunks or even more.
- The first problem was finding out how to design this message boundary.
- The solution was to invent a fixed 9-byte header:
    - 1 ASCII character for the message type (D, C, A), followed by 8 decimal digits for the payload length.
- Every call to frame the message prepends this header and every receive frame call loops until that many bytes have arrived.
- The problem occurred now when TCP read fewer than 9 bytes in a single recv call, so the solution was to lop until all bytes have been receive.

2. Changes to the Designed Architecture

- Initially, the plan was to use Websocket (WSS) + HTTPS REST. However, as per the assignment requirements, one had to create their own protocol. The changes were made to use raw TCP sockets plus custom framing (the 9-byte header protocol).

- Instead of MessagePack for message parsing, plain text was used, separated by colons, following the <command>:<recipient>:<content> format.

- Running PostgreSQl server for memory storage requires a separate process, network binding and driver dependencies. This was too much of work for this small project and could have ended up slowing down the server. The solution was to use SQLite in WAL mode, allowing connections to be shared across threading model and just required one file for storage. Though it doesn't scale beyond one machine, but it will suffice.

- For in-memory cache and user presence, Real Redis Server was initially in consideration. However, due to a hu8ge requirement setup, fakeredis was therefore used. Has no network overhead and it uses the same API.

- For reasons similar to PostgreSQL and Redis Server, Google Cloud Server (GCS) was not used, and the solution was to store base64 codes in SQLite BLOB, which are sent over TCP connection, eliminates the need for signed URLs.

- Instead of using WebRTC for browser-standard P2P calls, UDP and manual NAT hole punch was used for calls and signalling. The server will tell both clients each other's IP and UDP port, then both sides simultaneously fire PUNCH packets at each other. The NAT on each side opens a pinhole for packets arriving from the peer's address. When a PUNCH arrives, an ACK is fired back and the blocking wait event is unblocked and the two clients are connected.

- For concurrency implementation, the server spawns a new thread per client connection, which can lead to a potential race condition. Locks are therefore imposed:
    - db_lock to serialise all SQLite writes, while WASL mode still allows concurrent threads.
    - Client_Lock guards against user presence checks.
    - Threading.Lock() to guard against data races.

Video calls were just hard to implement, so as group calls, so the features were discarded. The server uses TCP for reliable connection and everything text-based, and uses UDP for fast, unordered live audio streaming. When a call is initiated, a separate UDP socket is created. This socket binds to a random available port on 0.0.0.0 and that port number is what gets registered with the server at login. During a call, raw audio chunks are wrapped in a 1-byte packet type header and fired over UDP directly peer-to-peer, bypassing the server entirely after the initial signalling phase.

Another TCP socket is also used to listen for incoming file transfers from peers. Only image, audio, video and pdf files can be transferred via the server. Issues occurred when receivers received corrupted files, which were mainly caused by the file wrapping the socket in a buffered reader that eargely reads ahead so some bytes from the actual file body were silently consumed into its internal buffer. This was fixed by using an HTTP server handler method rfile.read() to read raw body content.

5. Performance and Reliability Discussion

Text messaging is fast and reliable. The framed TCP protocol ensures messages arrive complete, and the Redis offline queue means nothing is lost when a user is offline. The 20-minute inactivity timeout keeps idle connections from accumulating on the server.

File transfers perform well for small to medium files since they go peer-to-peer and bypass the server entirely. Large files will be slower because they're fully base64-encoded before transmission, adding a bit overhead to every file, and the entire file must fit in a single framed message. Capping videos to 45 seconds was on of the ways to minimise the overhead.

Voice calls are functional for local or low-latency networks. Raw uncompressed PCM audio is streamed at 44100 Hz, which is high quality but bandwidth-heavy (~86 KB/s per call). There's no compression, so call quality will degrade noticeably on congested or high-latency connections.

Concurrency was handled reasonably well. SQLite in WAL mode allows concurrent reads alongside writes, and the per-client thread model keeps one slow client from blocking others. That said, a thread-per-client model won't scale gracefully beyond roughly 50–100 simultaneous users before thread overhead becomes a bottleneck.

The biggest practical limitation is *fakeredis*. It's an in-process mock that resets completely every time the server restarts, wiping all offline queues and presence data. However, for the scope of the assignment, it will suffice.

Finally, we used GenAI to generate a Readme file, making it easier for spectators to understand and go through the project.

6. Basic Security Consideration

Passwords are salted with a random 16-byte salt and hashed with SHA256, and hmac.compare_digest, with the help of GenAI. This prevents timing attacks on verification. A weak password blocklist and complexity rules are enforced at registration. Session inactivity timeouts limit the window for session hijacking.

However, all traffic, including passwords during login and all chat content travels in plaintext over TCP. Anyone on the same network can read every message and credential with a basic packet sniffer.

Authentication has no rate limiting or lockout on failed login attempts, leaving it open to brute-force attacks.

The framing protocol does no integrity checking. A message's declared length is trusted blindly.

Raw base64 file data in SQLite is stored with no content validation. A malicious user could upload arbitrary file types, and there's nothing preventing oversized uploads from bloating the database.

For small, trusted local networks, or academic projects, the system is slightly appropriate, however, a lot more needs to be done to build a larger-scale real world app like WhatsApp.

7. Use of Generative AI

GenAI was used to implement the whole user interface. The prototype was used as a template to transition from a console-based application to a UI based application. For security recommendations and server suggestions, GenAI was of great help. With the use of GenAI, we managed to discover *fakeredis* and *SQLite*, which helped us better implement our app.

Audio calls were hard to implement, as initially files were the ones transferred via UDP. However, a client wouldn't want to receive a half image, so TCP was the most reliable protocol for media transfers. In order to implement a hybrid structured system, we used GenAI to help implement audio call feature for peer-to-peer communication.